Advanced Artificial Intelligence: Part 2

Comprehensive Study Guide

April 28, 2026

Abstract

This document serves as a comprehensive study guide for the second part of the Advanced Artificial Intelligence course. It covers topics such as Fuzzy Logic, Intelligent Agents, State-Space Search, Local Search Algorithms, Constraint Satisfaction Problems (CSPs), Propositional and First-Order Logic, Bayesian Belief Networks, and Markov Decision Processes (MDPs).

Contents

1 Uncertain Reasoning and Fuzzy Logic **4**
1.1 Issues in Rule-Based Methods for Uncertain Reasoning 4
1.2 Fuzzy Logic: Handling Vagueness 4
1.2.1 Fuzzy Logic Master Formulas 4
1.3 Example: Cardiac Health Management (Numerical Tracing) 4

2 Introduction and Basics of Artificial Intelligence **5**
2.1 Human Cognition Abilities 5
2.2 Definitions of AI 5
2.3 The Turing Test (1950) 6
2.4 Evolution and Applications of AI 6

3 Constraint Satisfaction Problems (CSP) **6**
3.1 Core Components of a CSP 6
3.2 Constraint Propagation and Consistency 6
3.3 Backtracking Algorithm 6

4 AI Environments and Intelligent Agents **6**
4.1 Types of Intelligent Agents 6
4.2 Properties of Environments 7

5 Automated Problem Solving by Search **7**
5.1 Search Problem Formulation 7
5.2 Tower of Hanoi Example 7

6 Search Algorithms **8**
6.1 Blind (Uninformed) Search 8
6.2 Heuristic (Informed) Search: A* Algorithm 8

7 Local Search Algorithms **9**
7.1 Concept and Use Case 9
7.2 Iterative Deepening DFS (IDDFS) 9
7.3 Gradient Descent (Greedy Local Search) 9
7.4 The Trap: Local Minima 9

8 Escaping Local Minima **9**
8.1 Random Restart . . . 9
8.2 Simulated Annealing (Stochastic Search) . . . 10
8.3 Tabu Search . . . 10
8.4 Beam Search (Example: 3-SAT) . . . 10

9 Advanced Constraint Satisfaction Problems (CSP) **10**
9.1 CSP Solving Blueprint . . . 10
9.2 CSP Heuristics . . . 11
9.3 Backtracking Algorithm . . . 11

10 CSP Search Strategies and Limitations **11**
10.1 Backtracking vs. Forward Checking . . . 11
10.2 The Illusion of Forward Checking (3-Colouring Problem) . . . 11

11 Propositional Logic **12**
11.1 Deduction Steps (English to Math) . . . 12
11.2 Truth Tables and Natural Deduction . . . 12
11.3 Inadequacy of Propositional Logic . . . 12

12 First-Order Logic (FOL) and Skolemization **12**
12.1 Quantifiers and Golden Rules . . . 12
12.2 Skolemization: Eliminating $\exists$ . . . 13

13 Resolution Refutation **13**
13.1 Proof by Contradiction . . . 13

14 Handling Uncertain Knowledge **13**
14.1 Deduction vs. Induction . . . 13
14.2 The Failure of Classical Logic under Uncertainty . . . 13
14.3 Predicting vs. Diagnosing . . . 14

15 Axioms of Probability and Bayes' Rule **14**
15.1 Axioms of Probability . . . 14
15.2 Bayes' Rule . . . 14

16 Bayesian Belief Networks (BBN) **14**
16.1 Concept and Structure . . . 14
16.2 The Burglar Alarm Example . . . 14
16.3 Calculating Joint Probability . . . 15
16.4 Marginalization and Total Probability . . . 15

17 Advanced Bayesian Inference and Construction **15**
17.1 Types of Inferences . . . 15
17.2 Incremental Network Construction . . . 15
17.3 Rule-Based vs. Probabilistic Systems . . . 16

18 Fuzzy Logic **16**
18.1 The Need for Fuzzy Sets . . . 16
18.2 Fuzzy Rules and Evaluation . . . 16
18.3 Fuzzy Workflow (Example: Cardiac Risk) . . . 16

19 Markov Decision Processes (MDPs) 16
19.1 Stochastic Planning 16
19.2 Objective of an MDP 17
19.3 The Discount Factor (γ) 17
19.4 Solving MDPs 17
19.5 Value Iteration (Bellman Equation) 17

1 Uncertain Reasoning and Fuzzy Logic

1.1 Issues in Rule-Based Methods for Uncertain Reasoning

When attempting to incorporate probability into traditional logic (If-Then rules), three major issues arise. These issues highlight why probabilistic reasoning requires a holistic view of evidence rather than isolated logical deductions.

- **Locality:** In logical reasoning systems, if a rule $A \implies B$ exists and A is true, we can blindly conclude B without worrying about any other rules. In probabilistic systems, this is not possible. All available evidence must be considered (e.g., the "Explaining Away" concept where new evidence like an earthquake changes the probability of a burglary).
- **Detachment:** Once a logical proof is found for proposition B, we can use it regardless of how it was derived (it is detached from its justification). In probabilistic reasoning, the source of the evidence is crucial, and probabilities multiply and degrade sequentially.
- **Truth Functionality:** In logic, the truth of complex sentences (like $A \wedge B$) can be computed solely from the truth of the individual components. Probability combination does not work this way except under strong independence assumptions; calculating $P(A \wedge B)$ requires knowing the conditional relationship $P(A|B)$.

1.2 Fuzzy Logic: Handling Vagueness

While Probability deals with uncertainty (lack of information about an event happening), Fuzzy Logic deals with vagueness (imprecise definitions, like determining if a 90 kg man is "fat"). In Fuzzy Logic, truth is represented as a decimal value between 0 and 1.

1.2.1 Fuzzy Logic Master Formulas

The rules for evaluating the fuzzy truth (T) of a complex sentence are:

- **AND ($\wedge$):** Always take the minimum value. $T(A \wedge B) = \min(T(A), T(B))$
- **OR ($\vee$):** Always take the maximum value. $T(A \vee B) = \max(T(A), T(B))$
- **NOT ($\neg$):** Subtract from 1. $T(\neg A) = 1 - T(A)$

1.3 Example: Cardiac Health Management (Numerical Tracing)

Problem Statement: An AI system is designed for Cardiac Health Management using Fuzzy Logic. Given a patient with a daily diet of 3000 calories and exercise burning 1000 calories, calculate the exact percentage likelihood of heart disease risk.

Step 1: Fuzzification (Input Membership Functions)
The provided membership functions (fuzzification formulas) convert raw data into truth values between 0 and 1.

- Diet-High: $x/5000 \implies 3000/5000 = 0.6$
- Diet-Low: $1 - Diet_{High} \implies 1 - 0.6 = 0.4$
- Exercise-High: $y/2000 \implies 1000/2000 = 0.5$
- Exercise-Low: $1 - Exercise_{High} \implies 1 - 0.5 = 0.5$

Step 2: Rule Evaluation
Applying the Fuzzy Rules to determine the risk levels:

- **Rule 1 (Balanced):** IF Diet is Low AND Exercise is High $\implies$ Balanced. $T(Balanced) = \min(0.4, 0.5) = 0.4$. Thus, Risk-Low = 0.4.
- **Rule 2 (Unbalanced):** IF Diet is High OR Exercise is Low $\implies$ Unbalanced. $T(Unbalanced) = \max(0.6, 0.5) = 0.6$. Thus, Risk-High = 0.6.

Step 3: Graph Intersections (Clipping)
To determine where the risk curves plateau on the 0 to 100 percentage scale (z):

- Risk-Low Curve: $f(z) = 0.8 - (z/125)$. Limiting to 0.4 $\implies$ $0.4 = 0.8 - (z/125)$ $\implies$ $z = 50\%$.
- Risk-High Curve: $f(z) = z/125$. Limiting to 0.6 $\implies$ $0.6 = z/125$ $\implies$ $z = 75\%$.

Step 4: Defuzzification (Area Calculation)
The final percentage is found by calculating the Area Under the Curve (Aggregation using OR/MAX bounds) across three segments:

- **Segment 1 (0 to 50):** Rectangle of height 0.4. Area $= 50 \times 0.4 = 20$.
- **Segment 2 (50 to 75):** Trapezoid/Integral of the rising curve ($z/125$). $\int_{50}^{75} \frac{z}{125} dz = \frac{75^2 - 50^2}{250} = \frac{5625 - 2500}{250} = 12.5$.
- **Segment 3 (75 to 100):** Rectangle of height 0.6 and width 25. Area $= 25 \times 0.6 = 15$.

Final Likelihood: $20 + 12.5 + 15 = 47.5\%$. The patient has a 47.5% risk of heart disease.

2 Introduction and Basics of Artificial Intelligence

2.1 Human Cognition Abilities

- **Left Brain Functions:** Logic, analysis, sequencing, mathematics, language, facts. In AI, this represents "Symbolic methods" or Classical AI.
- **Right Brain Functions:** Creativity, imagination, holistic thinking, intuition, visualization. In AI, this maps to "Statistical methods" or Machine Learning.
- **Modern AI:** The convergence of both classical logic and modern statistical machine learning.

2.2 Definitions of AI

AI is defined across four quadrants. The modern curriculum explicitly follows the **"Act Rationally"** approach. This focuses on creating systems that take the best possible action to achieve a goal, bypassing philosophical debates about whether the machine is truly "conscious" or thinking like a human.

1. Systems that think like humans.
2. Systems that act like humans (e.g., Turing Test).
3. Systems that think rationally.
4. **Systems that act rationally** (The core modern AI paradigm).

2.3 The Turing Test (1950)

Proposed by Alan Turing to determine if a computer can demonstrate intelligence. A human interrogator communicates via a teletype with both a human and a computer. If the interrogator cannot distinguish the computer from the human based on the conversation, the computer passes the test.

2.4 Evolution and Applications of AI

A landmark event in AI history occurred on May 11, 1997, when IBM's "Deep Blue" defeated World Chess Champion Garry Kasparov by evaluating 200 million moves per second. Today, AI has greenfield applications in India across Healthcare (preventative medicine), Agriculture (rural development), Smart Mobility, Retail, Manufacturing, and Education.

3 Constraint Satisfaction Problems (CSP)

3.1 Core Components of a CSP

A Constraint Satisfaction Problem involves solving puzzles or scheduling tasks where certain rules must be obeyed. It comprises three elements:

- **Variables (X):** The entities whose values need to be found (e.g., empty Sudoku cells).
- **Domain (D):** The set of possible values each variable can take (e.g., numbers $1 - 9$).
- **Constraints (C):** The rules limiting the values (e.g., no duplicate numbers in a row).

3.2 Constraint Propagation and Consistency

To solve CSPs efficiently, AI attempts to reduce the domains of variables before brute-force guessing.

- **Node Consistency (Unary):** Ensuring a variable satisfies a rule applying only to itself (e.g., "Must be an even number").
- **Arc Consistency (Binary - AC-3):** Ensuring consistency between two variables. For example, if $A > B$ and domains are $\{1, 2, 3\}$, A cannot be 1 because no value in B is strictly less than 1. Thus, 1 is pruned from A's domain.

3.3 Backtracking Algorithm

When constraint propagation cannot reduce the domains to single values, the AI uses Backtracking. It assigns a value to a variable and moves forward. If a constraint is violated down the line, it backtracks, revokes the assignment, and tries an alternative value.

4 AI Environments and Intelligent Agents

4.1 Types of Intelligent Agents

1. **Simple Reflex Agent:** Operates solely on current "If-Then" rules without memory (e.g., a room heater).
2. **Model-based Reflex Agent:** Maintains an internal state (Model) to track aspects of the world it cannot currently see. Useful in partially observable environments.

3. **Goal-based Agent:** Considers future consequences by asking, "Will this action get me closer to my goal?"
4. **Utility-based Agent:** An upgrade over goal-based. If there are multiple paths to a goal, it chooses the optimal one that maximizes utility (e.g., quickest or cheapest route).
5. **Learning Agent:** Starts with a blank slate, acts, receives feedback (penalties/rewards), and learns to behave better over time (Reinforcement Learning).

4.2 Properties of Environments

The complexity of an AI problem depends entirely on the environment it operates in:

- **Fully vs. Partially Observable:** Can the AI see the entire state? (Chess is fully observable; Poker is partially observable).
- **Deterministic vs. Stochastic:** Is the next state strictly determined by the current action? (Chess is deterministic; Ludo is stochastic due to dice rolls).
- **Episodic vs. Sequential:** Do past actions affect future ones? (Image classification is episodic; Chess is sequential).
- **Static vs. Dynamic:** Does the environment change while the AI is thinking? (Crossword is static; Driving is dynamic).
- **Discrete vs. Continuous:** Are there a finite number of distinct states and actions? (Chess is discrete; Steering a car is continuous).

5 Automated Problem Solving by Search

5.1 Search Problem Formulation

When mathematical formulas fail, AI solves problems by searching through possible states. Every search problem comprises five key elements:

1. **Initial State:** Where does the problem start?
2. **Actions:** What legal moves can the AI take?
3. **Transition Model (Result):** What is the outcome of taking action A in state S?
4. **Goal Test:** Has the goal been reached?
5. **Path Cost:** What is the cumulative cost of the actions taken?

5.2 Tower of Hanoi Example

The Tower of Hanoi is a classic recursive search problem.

- **State Space:** Disks placed on 3 pegs. The state space is implicit; generating all 3^N states in advance would consume too much memory.
- **Constraints:** Only the topmost disk can be moved. A larger disk cannot be placed on a smaller one.

C++ Recursive Implementation:

```cpp
#include <iostream>
using namespace std;

void towerOfHanoi(int n, char source, char helper, char destination) {
    if (n == 1) { // Base Case
        cout << "Move disk 1 from " << source << " to " << destination
           << endl;
        return;
    }
    // Step 1: Move N-1 disks to Helper
    towerOfHanoi(n - 1, source, destination, helper);
    // Step 2: Move largest disk to Destination
    cout << "Move disk " << n << " from " << source << " to " <<
       destination << endl;
    // Step 3: Move N-1 disks from Helper to Destination
    towerOfHanoi(n - 1, helper, source, destination);
}

int main() {
    towerOfHanoi(3, 'A', 'B', 'C');
    return 0;
}
```

6 Search Algorithms

6.1 Blind (Uninformed) Search

- **Depth-First Search (DFS):** Explores one branch deeply before backtracking. Uses a **STACK** (LIFO). Memory efficient but not optimal.
- **Breadth-First Search (BFS):** Explores all nodes at the current depth before moving deeper. Uses a **QUEUE** (FIFO). Guarantees the shortest path but consumes massive memory.
- **Depth-Bounded DFS (DB-DFS):** Prevents infinite loops in DFS by setting a hard depth limit. If `depth < depthBound`, it generates children; otherwise, it backtracks.

6.2 Heuristic (Informed) Search: A* Algorithm

Blind searches lack direction. Heuristic searches utilize "additional knowledge" to estimate the distance to the goal, steering the search directionally.

The magic formula of A* is $f(n) = g(n) + h(n)$:

- $g(n)$: The exact path cost from the start node to the current node n.
- $h(n)$: The heuristic estimated cost from node n to the goal.

The OPEN and CLOSED Lists:

- **OPEN List:** Discovered nodes waiting to be explored. A* strictly picks the node with the lowest $f(n)$ from this list.
- **CLOSED List:** Fully explored nodes. This prevents the algorithm from cycling.

Crucial Stopping Condition: A* does **not** terminate the moment a goal node is generated. It only terminates when the goal node is pulled from the OPEN list for expansion. This guarantees that no cheaper alternative paths exist.

7 Local Search Algorithms

7.1 Concept and Use Case

In many optimization problems (like N-Queens or Time-Table Scheduling), the path taken to reach the goal is irrelevant; the goal state itself is the solution. Local search algorithms operate by maintaining only a single "current" state (or a small set of states) and iteratively attempting to improve it.

- **Memory Efficiency:** Highly efficient as they do not store the search tree or explored paths.
- **Optimization Objective:** The goal is to find a configuration that minimizes conflicts (or maximizes utility).

7.2 Iterative Deepening DFS (IDDFS)

IDDFS combines the space efficiency of DFS with the optimality of BFS. It repeatedly executes DFS, but with increasing depth limits (Limit = 1, then 2, then 3...). This prevents the AI from getting stuck in infinite paths while ensuring the shortest path is found.

7.3 Gradient Descent (Greedy Local Search)

Using the 8-Queens problem as an example, the heuristic value ($Value[state]$) is defined as the number of attacking pairs of queens. The algorithm attempts to minimize this value to 0.

- **Greedy Move:** The AI evaluates all immediate neighbor states and greedily selects the state that maximally reduces the number of conflicts (the steepest descent).

7.4 The Trap: Local Minima

Gradient descent suffers from a major flaw: getting stuck in **Local Minima**.

- **Local Minimum:** A state that is better than all its immediate neighbors, but is not the overall optimal state (Global Minimum). Once the AI reaches a local minimum, every possible move worsens the score, causing the AI to halt prematurely.
- **Flat Local Minimum / Plateau:** A flat region where all neighbor states have the same value, providing no gradient for the AI to follow.
- **Shoulder:** A flat region that eventually leads to further descent, but may temporarily trap a greedy algorithm.

8 Escaping Local Minima

8.1 Random Restart

A brute-force solution where, upon getting stuck, the AI completely abandons the current state and randomly generates a entirely new starting state. This is repeated many times, and the best result across all trials is returned.

8.2 Simulated Annealing (Stochastic Search)

Inspired by metallurgy (cooling metals slowly to increase strength), this algorithm occasionally allows "bad" moves (moves that worsen the state) to jump out of local minima.

- **The Acceptance Probability:** If a move improves the state ($\Delta E > 0$), it is always accepted. If a move worsens the state, it is accepted with a probability $P = e^{\Delta E/T}$.
- **Temperature (T):** Initially, T is high, making the AI act randomly and accept many bad moves. Over time, T is slowly reduced (cooled). As $T \to 0$, the AI becomes strictly greedy.

8.3 Tabu Search

Tabu search enhances local search by giving the AI a short-term memory to prevent infinite loops and cyclic loitering.

- **Tabu List:** A list of recently visited states or recently applied moves that are temporarily "banned" (Tabu).
- **Tabu Tenure:** The memory has a fixed size (e.g., $T = 3$). As new moves are added, the oldest banned moves expire and become valid again.
- **Mechanism:** By forbidding the AI from returning to recently visited states, it is forced to explore new, potentially worse paths, thereby escaping local minima.

8.4 Beam Search (Example: 3-SAT)

Beam search is an optimized search strategy that limits the number of states stored in memory to a fixed "Beam Width" (k).

- At each depth level, instead of exploring all possible children (like BFS), it only generates children from the top k best states.
- It aggressively prunes the search tree, saving massive amounts of memory while focusing only on the most promising paths.

9 Advanced Constraint Satisfaction Problems (CSP)

9.1 CSP Solving Blueprint

1. **Graph Creation:** Assign variables to nodes and draw edges representing constraints (e.g., in Airline Scheduling, conflicting flight times create an edge, meaning they cannot share a gate).
2. **Constraint Propagation:** Prune invalid options from variable domains before searching.
   - **Node Consistency:** Applying unary constraints.
   - **Arc Consistency:** Applying binary constraints. If a value in Variable A's domain has no valid matching value in Variable B's domain, remove the value from A.
3. **Search and Backtracking:** If propagation doesn't yield a single solution, use search heuristics to iteratively guess and backtrack.

9.2 CSP Heuristics

When the AI must guess a variable's value, it uses heuristics to minimize the chance of failure:

- **MRV (Minimum Remaining Values):** "Fail-first" strategy. The AI selects the variable with the fewest valid options remaining in its domain.
- **Degree Heuristic:** Used as a tie-breaker. Selects the variable involved in the largest number of constraints with other unassigned variables.
- **LCV (Least Constraining Value):** Once a variable is chosen, the AI assigns it the value that rules out the fewest choices for its neighboring variables.

9.3 Backtracking Algorithm

```
def CSP_BACKTRACKING(assignment):
    if assignment is complete:
        return assignment

    var = SELECT_UNASSIGNED_VARIABLE(assignment) # Using MRV/Degree

    for value in ORDER_DOMAIN_VALUES(var, assignment): # Using LCV
        if value is consistent with assignment:
            add {var = value} to assignment
            result = CSP_BACKTRACKING(assignment)
            if result != failure:
                return result
            remove {var = value} from assignment # Backtrack

    return failure
```

10 CSP Search Strategies and Limitations

10.1 Backtracking vs. Forward Checking

When solving a CSP, the AI can employ different predictive strategies:

- **Backtracking (Trial & Error):** The AI assigns a value and moves forward. If it hits a dead end (a constraint violation), it backtracks and changes the previous assignment. It is blind to future conflicts.
- **Forward Checking (Look-Ahead):** Whenever a variable X is assigned a value, Forward Checking immediately removes conflicting values from the domains of all unassigned variables Y connected to X. If any neighbor's domain becomes empty, the AI backtracks immediately, saving time.

10.2 The Illusion of Forward Checking (3-Colouring Problem)

While Forward Checking is faster than blind Backtracking, it has a critical limitation: **It does not detect all future inconsistencies.**

- **The Flaw:** Forward Checking only checks constraints between the *currently assigned* variable and its immediate unassigned neighbors. It does **not** check constraints between two unassigned neighbors.

- **Example:** If unassigned neighbors B and D both have only $\{Blue\}$ left in their domains, Forward Checking will not raise an alarm, even though assigning Blue to both will eventually cause a failure.
- **Solution: Maintaining Arc Consistency (MAC)** solves this by checking chain reactions among all unassigned variables, guaranteeing that early failures are caught.

11 Propositional Logic

11.1 Deduction Steps (English to Math)

AI evaluates the truth of human arguments by converting them into Boolean formulas:

1. **Variables (Atoms):** Assign letters (a, b, c) to English statements (e.g., $a =$ "I am the President").
2. **Connectors (Semantics):** Use logical operators: $\neg$ (NOT), $\wedge$ (AND), $\vee$ (OR), $\implies$ (IMPLIES).
3. **Codification:** Combine them into rules. "If I am the President, then I am well-known" becomes $a \implies b$.
4. **Combined Formula:** Combine all premises (ANDed together) implying the conclusion: $(Premise_1 \wedge Premise_2) \implies Conclusion$.

11.2 Truth Tables and Natural Deduction

To prove a formula is valid (a **Tautology**), we can use a Truth Table. If the final column is all True, the logic is flawless. However, Truth Tables are **NP-Hard** (an n-variable table has 2^n rows). To bypass this, AI uses **Natural Deduction** (Shortcut Rules):

- **Modus Ponens:** $(a \implies b), a \vdash b$
- **Modus Tollens:** $(a \implies b), \neg b \vdash \neg a$
- **Hypothetical Syllogism:** $(a \implies b), (b \implies c) \vdash (a \implies c)$
- **Disjunctive Syllogism:** $(a \vee b), \neg a \vdash b$

11.3 Inadequacy of Propositional Logic

Propositional logic fails when dealing with quantities, groups, or relationships. It cannot comprehend words like "All", "Some", or "Every" (e.g., "All dancers are graceful"). This necessitates **First-Order Logic (FOL)**.

12 First-Order Logic (FOL) and Skolemization

12.1 Quantifiers and Golden Rules

FOL introduces Predicates (e.g., $King(x)$) and Quantifiers:

- **Universal Quantifier ($\forall$ - For All):** Always paired with IMPLIES ($\implies$).
  Example: "All humans are mortal" $\implies \forall x(Human(x) \implies Mortal(x))$.
  (Using AND would incorrectly imply everything in the universe is human).
- **Existential Quantifier ($\exists$ - There Exists):** Always paired with AND ($\wedge$).
  Example: "Some student is smart" $\implies \exists x(Student(x) \wedge Smart(x))$.

12.2 Skolemization: Eliminating $\exists$

Computers cannot process the vague concept of "There Exists". **Skolemization** is the process of removing $\exists$ by assigning a concrete name to the entity.

1. **Skolem Constant:** If $\exists z$ is independent (not inside a $\forall$ loop), replace z with a fixed constant name (e.g., L).
   Example: "There exists a library" $\implies Library(L)$.

2. **Skolem Function:** If $\exists y$ is trapped inside $\forall x$, the entity y depends on x. Replace y with a function $f(x)$.
   Example: $\forall x \exists y (Mother(y, x)) \implies \forall x (Mother(f(x), x))$. We cannot use a constant, as that would imply everyone shares the exact same mother!

13 Resolution Refutation

13.1 Proof by Contradiction

To prove that a formula $F = (Premises \implies Goal)$ is valid, AI attempts to prove that its opposite ($\neg F$) is **unsatisfiable** (a contradiction).

- **Step 1 (Negation):** We assert the premises are true, but the goal is false: $Premises \wedge \neg Goal$.

- **Step 2 (CNF Conversion):** Convert all formulas into **Conjunctive Normal Form (CNF)** using substitution rules like Implication Elimination: $(a \implies b) \equiv (\neg a \vee b)$.

- **Step 3 (Resolution):** The Resolution Rule states that $(a \vee b)$ and $(\neg a \vee c)$ resolves to $(b \vee c)$.

- **Step 4 (False Derivation):** Continuously resolve clauses. If you eventually derive two contradictory clauses (a and $\neg a$), they resolve to **False**. This contradiction proves the original argument was valid.

14 Handling Uncertain Knowledge

14.1 Deduction vs. Induction

- **Deduction:** Associated with formal logic. It involves reasoning from known premises to a conclusion that is **certain** and inevitable.

- **Induction:** Associated with everyday arguments and statistical reasoning. The conclusions are probable and plausible, but **uncertain**.

14.2 The Failure of Classical Logic under Uncertainty

First-Order Logic demands exception-less rules, which is impractical in the real world.

- Example: $\forall p\, Symptom(p, Toothache) \implies Disease(p, Cavity)$.
  This is incorrect because a toothache can be caused by gum disease, an impacted wisdom tooth, etc. To make the rule 100% true in logic, the AI would have to list *every possible cause* in the universe, which is computationally impossible (Theoretical and Practical ignorance).

- Hence, AI shifts to Probability and statistical inference.

14.3 Predicting vs. Diagnosing

Probabilistic reasoning works in both directions:

- **Predicting (Cause to Effect):** Given that a patient has a cavity, what is the chance they will have a toothache?
- **Diagnosing (Effect to Cause):** Given that a patient has a toothache, what is the probability it is caused by a cavity?

15 Axioms of Probability and Bayes' Rule

15.1 Axioms of Probability

1. All probabilities are between 0 and 1: $0 \leq P(A) \leq 1$.
2. $P(True) = 1$ and $P(False) = 0$.
3. $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

15.2 Bayes' Rule

Bayes' Theorem is the cornerstone of AI diagnosis. It allows us to calculate $P(Disease|Symptom)$ if we know the prior probabilities.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

16 Bayesian Belief Networks (BBN)

16.1 Concept and Structure

A Belief Network is a Directed Acyclic Graph (DAG) used to represent probabilistic relationships among variables, overcoming the computational nightmare of calculating full joint distributions manually.

- **Nodes:** Random variables (e.g., Burglary, Alarm).
- **Directed Links (Arrows):** Represent direct probabilistic influence (causality).
- **Conditional Probability Table (CPT):** Each node has a table quantifying the effect its parents have on it.

16.2 The Burglar Alarm Example

- **Burglary (B)** and **Earthquake (E)** are independent root nodes.
- **Alarm (A)** depends directly on both B and E.
- **JohnCalls (J)** and **MaryCalls (M)** depend directly on A.

Crucial Property (Conditional Independence): If the state of the Alarm is known, John's decision to call is completely independent of Mary's decision, and also independent of whether a Burglary or Earthquake actually occurred. The Alarm node "blocks" the influence.

16.3 Calculating Joint Probability

The core formula for the Joint Probability Distribution in a BBN is the product of the probabilities of each node given its parents:

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i | Parents(x_i))$$

Example Calculation: What is the probability that there is no burglary ($\neg B$), no earthquake ($\neg E$), the alarm sounds (A), and both John (J) and Mary (M) call?

$$\begin{aligned} P(J \wedge M \wedge A \wedge \neg B \wedge \neg E) &= P(J|A) \times P(M|A) \times P(A|\neg B, \neg E) \times P(\neg B) \times P(\neg E) \\ &= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998 \\ &= 0.00062 \end{aligned}$$

16.4 Marginalization and Total Probability

To find the probability of a single event (e.g., $P(A)$) when its parents (B, E) are unknown, the AI must sum over all possible combinations of the parents:

$$\begin{aligned} P(A) &= P(A \wedge B \wedge E) + P(A \wedge B \wedge \neg E) + P(A \wedge \neg B \wedge E) + P(A \wedge \neg B \wedge \neg E) \\ &= P(A|B, E)P(B)P(E) + \ldots \end{aligned}$$

17 Advanced Bayesian Inference and Construction

17.1 Types of Inferences

Bayesian Networks can perform reasoning in multiple directions:

- **Diagnostic Inference (Effect to Cause):** Given an effect, what is the probability of its cause? (e.g., Given Alarm, infer Burglary).
- **Causal Inference (Cause to Effect):** Given a cause, what is the probability of its effect? (e.g., Given Burglary, infer JohnCalls).
- **Inter-causal Inference (Explaining Away):** Inference between two causes of a common effect. If the Alarm sounds, the probability of Burglary is high. But if we then discover an Earthquake occurred, the probability of Burglary drops significantly because the Earthquake "explains away" the Alarm.

17.2 Incremental Network Construction

To build a Bayesian Network from scratch:

1. **Choose Variables:** Identify all relevant nodes.
2. **Ordering:** Crucially, order variables such that causes precede effects.
3. **Add Nodes & Links:** For each node, select a minimal set of parents that directly influence it, satisfying the conditional independence property.
4. **Define CPTs:** Create the Conditional Probability Table for each node.

17.3 Rule-Based vs. Probabilistic Systems

Early AI used Rule-Based Logic (e.g., MYCIN's Certainty Factors), which failed under uncertainty due to:

- **Locality:** Logic ignores global context once a local rule triggers. Probability considers all available evidence globally.
- **Detachment:** Logic proves a fact and forgets the source. Probability tracks the source to prevent double-counting evidence.
- **Truth Functionality:** Logic blindly evaluates $P \wedge Q$ from their individual truths. Probability requires analyzing the *dependency* between P and Q.

18 Fuzzy Logic

18.1 The Need for Fuzzy Sets

In classical logic, truth is binary (0 or 1). In Fuzzy Logic, truth is a **degree of membership** between 0 and 1, representing continuous concepts like "fat" or "tall".

18.2 Fuzzy Rules and Evaluation

Fuzzy Logic uses specific operators to evaluate complex rules:

- **AND (Intersection):** $T(A \wedge B) = \min(T(A), T(B))$
- **OR (Union):** $T(A \vee B) = \max(T(A), T(B))$
- **NOT (Complement):** $T(\neg A) = 1 - T(A)$

18.3 Fuzzy Workflow (Example: Cardiac Risk)

1. **Fuzzification:** Convert crisp inputs (e.g., 3000 calories) into fuzzy values using membership graphs.
   Example: $Truth(Diet_High) = 0.6$, $Truth(Diet_Low) = 0.4$.
2. **Inference:** Apply rules.
   "Diet is low AND Exercise is high $\implies$ Balanced" becomes $\min(0.4, 0.5) = 0.4$.
3. **Defuzzification:** Convert the fuzzy result back into a single crisp number. This is done by aggregating the risk functions (combining the curves for Risk-Low and Risk-High) and calculating the **Center of Gravity** (using integration/calculus to find the area under the combined curve).

19 Markov Decision Processes (MDPs)

19.1 Stochastic Planning

Unlike traditional search where actions have guaranteed outcomes, MDPs handle **Stochastic Environments** where actions have probabilistic outcomes (e.g., trying to move forward might result in slipping to the side).

19.2 Objective of an MDP

The goal is to find an optimal **Policy (π)**.

- A policy is a mapping from States to Actions ($\pi : S \to A$).
- It acts as a universal manual: "If you are in state S, take action A to maximize expected reward."

19.3 The Discount Factor (γ)

The total reward is calculated as: $R = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$

- γ (between 0 and 1) discounts future rewards. It ensures that infinite sums converge to a finite number and makes the AI prefer immediate rewards over distant, uncertain ones.

19.4 Solving MDPs

- **Acyclic MDPs:** Can be solved simply by working backward from the goal.
- **Cyclic MDPs (Loops):** Require solving a System of Linear Equations.
  - **Approach 1 (Exact Solution):** Solves $|S|$ equations simultaneously. Has a time complexity of $O(|S|^3)$, which crashes for large state spaces.
  - **Approach 2 (Iterative Policy Evaluation):** An engineering shortcut. Start by guessing all values as 0. Iteratively update the value of each state based on the current values of its neighbors.

$$V_n^\pi(s) = \sum_{s'} T(s, \pi(s), s')[C(s, \pi(s), s') + V_{n-1}^\pi(s')]$$

The process halts when the values converge (ϵ-consistency).

19.5 Value Iteration (Bellman Equation)

To find the *best* policy, AI uses the Bellman Equation to evaluate all possible actions at each step and greedily select the one with the maximum expected utility:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$$